

ERPNext as Backend vs Custom Backend

SmartStore ERP | Decision Analysis for Kartik (Alphenta)

Your Current Situation

You have a **7,034-line HTML frontend** with **18 modules**, beautiful glassmorphism UI rated **8/10**. All 10 transformation phases complete. Marathi + English bilingual. But the product behind it scores **2/10** — zero backend, no database, no real CRUD. Data lives in localStorage and disappears on browser clear.

Target market: Small malls and wholesale buyers in India, under 50 crore revenue.

What you need: Database, authentication, real CRUD APIs, offline/PWA, Tally export, UPI integration, role-based access, and more (19 gaps identified in gap analysis).

Option 1: ERPNext as Backend

Install ERPNext (free, open source), configure it for FMCG retail, and connect your HTML frontend to its REST APIs.

What ERPNext Gives You (Good)

- ✓ Free and open source — no license cost
- ✓ 15+ years mature, used by thousands of companies worldwide
- ✓ Built-in accounting, GST, inventory, HR, payroll — ready out of the box
- ✓ Large community, lots of tutorials and documentation
- ✓ Self-hostable on your own server

What ERPNext Costs You (Bad)

- ✗ **Your UI dies.** ERPNext has its OWN frontend (Frappe UI). To use your HTML, you fight the framework constantly. Most people give up and use ERPNext's default UI — your 10 phases of work wasted.
- ✗ **Heavy.** Needs minimum 4GB RAM server (Rs 1,500-3,000/month hosting). Your small mall clients won't pay that.
- ✗ **Steep learning curve.** Frappe framework has its own way of doing things (DocTypes, hooks, whitelisted methods). Takes 2-4 weeks just to understand.
- ✗ **Overkill.** ERPNext is designed for mid-size companies. For a small mall owner billing Rs 5 lakh/month, it's a truck to carry a bag.
- ✗ **Customization is painful.** Changing billing logic = editing Frappe DocTypes, custom scripts, their ORM. Every upgrade can break your customizations.
- ✗ **No offline by default.** ERPNext is server-dependent. Tier-2/3 cities with bad internet = dead ERP.

Your 18 Modules vs ERPNext Equivalents

Your Module	ERPNext DocType	Status
Dashboard KPIs	Custom API calls to multiple DocTypes	Partial
POS Billing	POS Invoice	Available

Your Module	ERPNext DocType	Status
Orders	Sales Order / Sales Invoice	Available
Purchase	Purchase Order / Purchase Receipt / Purchase Invoice	Available
Inventory	Stock Entry, Stock Ledger, Warehouse	Available
Expiry & Wastage	Batch (has expiry) + Stock Entry (wastage)	Partial
Schemes & Offers	Pricing Rule	Basic only
Dealers / Udhaar	Customer + Payment Entry (outstanding)	Partial
Payments	Payment Entry	Available
Staff Payroll	Salary Slip, Payroll Entry	Available
Data Hub	No direct match	Custom needed
GST e-Invoice	ERPNext India module (built-in)	Available
Bank Reconciliation	Bank Reconciliation Tool	Available
Fraud Detection	Does NOT exist	Custom needed
India Law Compliance	Partially built-in (GST only)	Partial
WhatsApp Chatbot	Does NOT exist	Custom needed
AI Demand Forecast	Does NOT exist	Custom needed
Barcode Scanner	POS has basic barcode support	Basic only

5 of your 18 modules have NO ERPNext equivalent — you'd build those from scratch anyway.

API Calls Needed Per Dashboard KPI Card

Every single KPI card on your dashboard needs a separate API call to ERPNext. Here's what each one requires:

KPI Card	ERPNext API Endpoint
Today's Sales	GET /api/method/erpnext.selling.report.sales_analytics
Monthly Revenue	GET /api/resource/Sales Invoice?filters=[[posting_date>=...]]
Stock Value	GET /api/method/erpnext.stock.utils.get_stock_value_on
Pending Orders	GET /api/resource/Sales Order?filters=[[status=To Deliver]]
Udhaar Pending	GET /api/resource/Payment Entry?filters=[[outstanding>0]]
Expiring Items	GET /api/method/...batch.get_batch_qty + custom filter
Staff Present	Custom API (no attendance KPI built-in)
GST Due	Custom API (not available)

For EACH of your 18 screens, you'd need to: (1) Find the right ERPNext API endpoint, (2) Understand its response format, (3) Write JavaScript fetch() calls, (4) Transform ERPNext's response into what your UI expects, (5) Handle auth tokens, errors, pagination. Estimated: **50-80 custom API integrations total**.

What ERPNext Cannot Provide

- ✗ **Real-time dashboard updates** — ERPNext has no WebSocket for live KPIs
- ✗ **Marathi bilingual data** — ERPNext stores data in one language only
- ✗ **AI anything** — zero AI built-in, all custom code needed
- ✗ **WhatsApp integration** — needs third-party (Twilio/Gupshup) + custom code
- ✗ **Fraud detection** — fully custom build required
- ✗ **Your glassmorphism UI** — ERPNext API gives raw JSON, your JS does all rendering
- ✗ **Offline/PWA** — no service worker, no offline support by default

Option 2: Claude Builds Custom Backend

A clean API backend (FastAPI or Express.js) + database (PostgreSQL/SQLite) that your existing HTML frontend calls via fetch(). Your UI stays exactly as-is.

Advantages

- ✓ **Your UI stays 100% intact.** All 18 modules, Marathi, glassmorphism — untouched.
- ✓ **Lightweight.** Can run on a Rs 500/month server or even a Raspberry Pi.
- ✓ **Fast to build.** No learning curve for you. Claude writes it, you test it.
- ✓ **Perfectly matched.** Every API endpoint maps exactly to your 18 modules. No bloat.
- ✓ **Easy to add features.** Tally export? One endpoint. UPI? One more. No framework fighting.
- ✓ **Offline/PWA ready.** Service workers can be added for offline support.
- ✓ **You own everything.** No dependency on ERPNext's roadmap or breaking changes.
- ✓ **Indian-specific from day 1.** GST, Tally, UPI, WhatsApp — built exactly for your market.

Limitations

- Not battle-tested like ERPNext — new code, may have initial bugs
- Accounting logic built from scratch (double-entry, ledger, trial balance)
- Long-term maintenance requires someone who understands the code
- Basic server knowledge needed for deployment

Head-to-Head Comparison

Factor	ERPNext	Custom Backend
Your effort required	HIGH — learn Frappe, configure, fight UI	LOW — test and give feedback only
Your UI survives?	NO — ERPNext forces its own frontend	YES — 100% intact
Time to working product	4-8 weeks (learning + configuring)	1-2 weeks per module
Server cost	Rs 1,500-3,000/month minimum	Rs 500/month or less
Offline support	No (needs heavy workarounds)	Yes (PWA ready)
Fits small mall clients?	Overkill for small businesses	Perfect fit
GST / Tally / Indian	Built-in but rigid	Built exactly how you want
Maturity	15+ years, battle-tested	New, needs testing
Long-term risk	ERPNext community dependency	Code maintainability
Total cost	Free license + expensive server	Free (Claude) + cheap server

Effort & Timeline Summary

Task	ERPNext Route	Custom Backend Route
Server setup	1 week (Linux, MariaDB, Redis, Node)	1 day (simple server)
Learn framework	2-4 weeks (Frappe)	0 — Claude does it
Connect 18 modules	4-6 weeks (50-80 API calls)	1-2 weeks (exact match APIs)
Build missing 5 modules	4-6 weeks (custom Frappe code)	Same effort either way
Auth & security	Built-in (but rigid roles)	1-2 days (custom, flexible)
Testing & debugging	Ongoing (two systems to debug)	Simpler (one codebase)
TOTAL	2-3 months + ongoing pain	3-5 weeks + clean codebase

VERDICT

Custom backend built by Claude is far more convenient.

- ➔ 1. You already did the hard part — 10 phases of UI work across 18 modules. ERPNext throws that away. Custom backend preserves it.
- ➔ 2. You're a founder, not a Frappe developer — your time is better spent on product decisions, sales, and client acquisition than learning a framework.

- ➔ **3. Your customers are small malls** — they need lightweight, offline-capable, cheap to run. ERPNext is none of those things.
- ➔ **4. 5 modules don't exist in ERPNext anyway** — AI Forecast, WhatsApp, Fraud Detection, Data Hub, and full India Law Compliance need custom code regardless.
- ➔ **5. Speed to market** — 3-5 weeks vs 2-3 months. First-mover advantage in the Indian small-mall FMCG ERP space matters.

ERPNext makes sense ONLY if you had no frontend and wanted an out-of-the-box solution. But you're past that point — you have a beautiful, India-specific, bilingual frontend. Plugging a custom backend into it is the smarter move.